

Kneo Agent Client

User Guide

Version 0.4.0

2026-05-27

Quickstart, profiles and auth, idempotency and retries, pagination, error handling, and the compatibility matrix — one printable reading path for the kneo-client user documentation.

Table of contents

Table of contents	2
Quickstart	4
What "quickstart" means in kneo-client	4
Install	4
Configure a profile	4
Ping the platform	5
Create your first run	6
Sync facade	7
One-shot: <code>asyncio.run()</code>	7
Repeated calls: <code>anyio.from_thread.start_blocking_portal()</code>	7
Where to go next	8
Profiles and auth	9
What "profile" means in kneo-client	9
Resolution order	9
TOML format	10
Environment variables	10
Auth schemes — what the platform accepts	11
Multi-profile workflows	11
Inspecting a resolved profile	12
Profile errors	13
Default config path	13
Idempotency and retries	15
What "idempotency and retries" mean in kneo-client	15
Idempotency keys, by default	15
When to supply your own key	16
Catching 409 mismatch	16
The retry policy	17
When retries fire	17
Customizing the policy	18
What surfaces when retries exhaust	19
Bypassing the auto-injection	19
Putting it together — a robust pattern	19
List-method results	21
Per-method return shapes	21
Page[T]	22
Iterating one page	22
Walking all pages: <code>iterate_all()</code>	22

The audit case (limit-only, gracefully degraded)	
Choosing a page_size	23
Map[K, V]	24
Filters + paging	25
Drop to the raw response	25
Auto-walking is not on the roadmap	25
Polling and waiting	27
wait_for_completion()	27
tail_trace()	27
When to use which	28
Polling cadence	29
Custom terminal statuses	29
TimeoutError handling	29
Error handling	31
What "errors" mean in kneo-client	31
Hierarchy	31
What every exception carries	32
Status code → exception mapping	32
Catching patterns	33
Catch broadly, log richly	33
Branch on specific status	33
Handle idempotency-key mismatches loudly	34
Don't catch successful-status branches	34
Transport errors and network troubleshooting	35
Timeouts	35
TLS verification	36
HTTP proxies	36
Inspecting the underlying httpx exception	37
Building error messages for users	37
Why a typed hierarchy	38
Reference	38
Compatibility matrix	39
What "compatibility" means in kneo-client	39
Current matrix	39
How pinning works in practice	39
Forward compatibility — newer kneo_serv than the pin	40
Backward compatibility — older kneo_serv than the pin	40
Breaking changes	41
Verifying compatibility yourself	41
What the pin does not guarantee	42

Quickstart

Source: <docs/user/quickstart.md>

This guide walks you from a fresh shell to your first authenticated call against a running Kneo Agent Platform instance, then to creating a run and inspecting its trace.

What "quickstart" means in kneo-client

`kneo-client` is a typed Python SDK and adapter toolkit for the Kneo Agent Platform's `/v1` HTTP API. Two products use it as their shared client layer — Kneo Agent Dashboard for operations and Kneo Agent Studio for development — but you can use it directly from any Python 3.12+ codebase that needs to talk to a platform instance.

A working setup needs three things:

1. The `kneo-client` package installed.
2. A **profile** — a `(url, api_key, auth_scheme, timeout)` tuple resolved from a TOML config file, environment variables, or explicit kwargs. See [Profiles and auth](#) for the full resolution model.
3. A reachable platform instance.

The rest of this page assumes you have all three.

Install

```
python -m pip install kneo-client
```

Requires Python ≥ 3.12 (3.12 / 3.13 / 3.14 tested in CI). The runtime closure is small: `httpx`, `pydantic`, `anyio`, `platformdirs`, `attrs`. No CLI is installed — `kneo-client` is a library, not a service.

Verify the install:

```
python -c "from kneo_client import KneoClient, __version__; print(__version__)"
# → 0.4.0
```

Configure a profile

The client reads connection details in this order (later sources override earlier):

1. A TOML config file at `~/.config/kneo/client.toml` (XDG-style location via [platformdirs](#)).
2. Environment variables: `KNEO_PROFILE`, `KNEO_URL`, `KNEO_API_KEY`, `KNEO_AUTH_SCHEME`, `KNEO_TIMEOUT`.
3. Explicit keyword arguments to `KneoClient.from_profile()` (or the underlying `load_profile()`).

The minimum-friction setup is env vars — useful for one-off scripts and CI:

```
export KNEO_URL=https://kneo.example.com
export KNEO_API_KEY=your-api-key
```

For named multi-profile setups (e.g. `default` for prod, `staging` for non-prod), use the TOML file:

```
# ~/.config/kneo/client.toml
[default]
url = "https://kneo.example.com"
api_key = "prod-key"
auth_scheme = "bearer"
timeout = 30.0

[staging]
url = "https://staging-kneo.example.com"
api_key = "staging-key"
```

Switch profiles by name (`KneoClient.from_profile("staging")`) or by env (`KNEO_PROFILE=staging`).

See [Profiles and auth](#) for the full resolution semantics, both auth schemes, and the multi-profile workflow.

Ping the platform

The smallest possible script — verifies that auth flows correctly and the platform is reachable:

```
import asyncio
from kneo_client import KneoClient

async def main():
    async with KneoClient.from_profile() as client:
        ready = await client.platform.health.readyz()
        print(f"platform ready: ok={ready.ok} version={ready.service!r}")
```

```
asyncio.run(main())
```

What this exercises:

- Profile resolution (TOML / env / kwargs).
- API-key injection on the outgoing request (`Authorization: Bearer ...` or `X-Kneo-API-Key: ...` depending on scheme).
- The transport's retry loop (will retry on transient network failures).
- Error mapping (a missing key surfaces as `KneoAuthError`, an unreachable URL as `KneoNetworkError`).

If you get a `KneoAuthError`, your API key isn't reaching the platform — re-check the profile resolution chain. If you get a `KneoNetworkError`, the URL is unreachable; see [Error handling & Transport errors and network troubleshooting](#) for timeouts, TLS, proxies, and how to inspect the underlying httpx cause.

Create your first run

A run is the unit of work the platform executes — it instantiates a spec (an agent definition) and tracks its lifecycle through `queued` → `running` → `terminal` {`completed`, `failed`, `cancelled`}. To create one:

```
async with KneoClient.from_profile() as client:
    created = await client.platform.runs.create({"spec_id": "your-spec-id"})
    print(f"run_id={created.run_id} status={created.status}")

    terminal = await client.platform.runs.wait_for_completion(
        created.run_id, poll_interval=2.0, timeout=600
    )
    print(f"final status: {terminal.status}")

    trace = await client.platform.runs.trace(created.run_id, limit=20)
    for event in trace.events:
        print(event)
```

The interesting parts:

- `runs.create(...)` auto-injects an `Idempotency-Key` header (UUID4). Re-running the same payload with the same key is safe — the platform replays the original response. See [Idempotency and retries](#).
- `runs.wait_for_completion(...)` polls `runs.get(run_id)` until the run reaches a terminal status. Default terminal set is `{"completed", "failed", "cancelled"}`; pass `terminal_statuses={"paused_human_review", ...}` to treat additional states as terminal.

- `runs.trace(...)` returns events the platform recorded during the run (tool calls, model calls, middleware decisions, policy checks, etc.).

This is the operational core of the platform adapter. Every other endpoint follows the same shape: `client.platform.<resource>.<method>(...)` returning a typed model.

Sync facade

If your caller can't run an event loop (a script, a notebook, a sync framework), wrap the transport with `SyncTransport`:

```
from kneo_client.core.profiles import load_profile
from kneo_client.core.transport import SyncTransport

with SyncTransport(load_profile()) as transport:
    response = transport.request("GET", "/v1/healthz")
    print(response.json())
```

Under the hood `SyncTransport` runs `Transport` inside an `anyio.from_thread.start_blocking_portal()` — same retry / idempotency / error flows, called synchronously. The async surface is the recommended path; the sync facade is for ergonomics.

`SyncTransport` does **not** mount `.platform` / `.agent` adapters — those are async-only. Two patterns work for sync consumers who want the wrapped endpoints:

One-shot: `asyncio.run()`

If you're calling once and not in an existing event loop:

```
import asyncio
from kneo_client import KneoClient

async def fetch() -> str:
    async with KneoClient.from_profile() as client:
        run = await client.platform.runs.create({"spec_id": "s1"})
        return run.run_id

run_id = asyncio.run(fetch())
print(run_id)
```

Repeated calls: `anyio.from_thread.start_blocking_portal()`

If you're calling many times from sync code (e.g., inside a synchronous CLI loop) and want to avoid the startup cost of `asyncio.run()` per call, hold open a portal — the same machinery `SyncTransport`

itself uses:

```
from anyio.from_thread import start_blocking_portal
from kneo_client import KneoClient

with start_blocking_portal() as portal:
    client = portal.call(KneoClient.from_profile().__aenter__)
    try:
        run = portal.call(client.platform.runs.create, {"spec_id": "s1"})
        status = portal.call(client.platform.runs.get, run.run_id)
        print(status.status)
    finally:
        portal.call(client.__aexit__, None, None, None)
```

The portal runs an event loop on a background thread; `portal.call(async_fn, *args)` blocks the calling thread until the coroutine completes. This is the recommended pattern for long-lived sync code that talks to the platform repeatedly.

Where to go next

By topic:

- [Profiles and auth](#) — multi-profile workflows, the two header schemes the platform supports, environment-variable precedence.
- [Idempotency and retries](#) — when keys are auto-injected, how 409 mismatches surface, customizing the retry policy.
- [List-method results](#) — `Page[T]` and `Map[K, V]` wrappers, walking pages with `iterate_all()`, the audit limit-only case.
- [Error handling](#) — the full exception hierarchy and what to catch.
- [Compatibility matrix](#) — which `kneo-client` releases support which `kneo_serv` versions.

For the comprehensive API reference (every class, method, exception), see the [API Reference HTML](#) or the [PDF version](#).

For runnable end-to-end scripts: [examples/](#).

Profiles and auth

Source: docs/user/profiles_and_auth.md

This guide explains how `kneo-client` resolves connection details (URL, API key, auth scheme, timeout) into a profile, what the two API-key header schemes mean, and how to set up multi-profile workflows for dev / staging / prod.

What "profile" means in kneo-client

A **profile** is a frozen dataclass bundling `(name, url, api_key, auth_scheme, timeout)`:

```
@dataclass(frozen=True)
class Profile:
    name: str
    url: str
    api_key: str
    auth_scheme: AuthScheme = AuthScheme.BEARER
    timeout: float = 30.0
```

Everything `Transport` needs to talk to one Kneo Agent Platform instance fits in those five fields. A `KneoClient` is built around exactly one profile; you construct one per platform instance you talk to.

Profiles are typically named — `default`, `staging`, `prod`, `ci`, etc. The name is informational (it appears in log lines) but doubles as the section name in the TOML config file.

Resolution order

`load_profile()` and `KneoClient.from_profile()` merge values from three sources, with later sources overriding earlier ones:

1. **TOML config file**, by default `~/.config/kneo/client.toml` (XDG-style via [platformdirs](#)). Pass `config_file=Path(...)` to point at a different file. If the file doesn't exist, it's skipped silently — env vars and explicit kwargs are still consulted.
2. **Environment variables**: `KNEO_URL`, `KNEO_API_KEY`, `KNEO_AUTH_SCHEME`, `KNEO_TIMEOUT`. (`KNEO_PROFILE` selects which TOML section to load — it does *not* override field values.)
3. **Explicit keyword arguments** to the function call.

If `url` or `api_key` cannot be resolved from any source, a `ProfileError` is raised with details on which sources were checked. Bad TOML, an unknown `auth_scheme`, or a non-numeric `timeout` also surface as `ProfileError`.

```

from kneo_client.core.profiles import load_profile

p = load_profile()                # 'default' from TOML + env
p = load_profile("staging")       # explicit profile name
p = load_profile(url="https://ad-hoc", api_key=token) # explicit kwargs win

```

TOML format

Each top-level table is one profile:

```

# ~/.config/kneo/client.toml
[default]
url = "https://kneo.example.com"
api_key = "prod-key"
auth_scheme = "bearer"          # or "kneo_api_key"
timeout = 30.0                  # seconds

[staging]
url = "https://staging-kneo.example.com"
api_key = "staging-key"

[local]
url = "http://127.0.0.1:8000"
api_key = "dev-token"
auth_scheme = "kneo_api_key"

```

`auth_scheme` and `timeout` are optional; their defaults are `"bearer"` and `30.0`.

Picking a profile at call time:

```

client = KneoClient.from_profile()    # 'default' (or $KNEO_PROFILE)
client = KneoClient.from_profile("staging") # explicit
client = KneoClient.from_profile("local")  # explicit

```

Environment variables

Variable	Purpose
<code>KNEO_PROFILE</code>	Profile name to load. Falls back to <code>"default"</code> if unset.
<code>KNEO_URL</code>	Override the profile's URL.

Variable	Purpose
<code>KNEO_API_KEY</code>	Override the profile's API key.
<code>KNEO_AUTH_SCHEME</code>	Override the scheme. Accepts <code>"bearer"</code> or <code>"kneo_api_key"</code> .
<code>KNEO_TIMEOUT</code>	Override the per-request timeout (float seconds).

CI environments typically set just `KNEO_URL` and `KNEO_API_KEY` and skip the TOML file entirely. The lack of a config file is *not* an error — env vars + kwargs can satisfy resolution on their own.

A bad value for `KNEO_TIMEOUT` (non-numeric) raises `ProfileError` with the variable name in the message — easier to debug than a silent fallback.

Auth schemes — what the platform accepts

The platform accepts the API key in either of two header schemes. They are *semantically* equivalent — both end up at the same platform code path — but operationally they have different trade-offs:

Scheme	Header sent	When to choose
<code>bearer</code> (default)	<code>Authorization: Bearer <key></code>	Works with most reverse proxies. Easy to revoke at the gateway layer. Standard HTTP semantics.
<code>kneo_api_key</code>	<code>X-Kneo-API-Key: <key></code>	Useful when your edge stack already uses the <code>Authorization</code> header for something else (mutual TLS auth, an upstream OAuth flow, etc.). Avoids the collision.

When in doubt, start with `bearer`. You can switch schemes per-profile without code changes — just update the TOML or the env var.

The two schemes are implemented by `kneo_client.core.auth.ApiKeyAuth`, an `httpx.Auth` subclass that injects whichever header the active profile selects. Internally the auth flow runs *inside* `httpx`'s request flow (after `Transport` has added its other headers), so the API key reaches every redirect / retry attempt at the right layer.

Multi-profile workflows

A common pattern in CI / local dev:

```
import os
from kneo_client import KneoClient

profile_name = "ci" if os.getenv("CI") else "default"
async with KneoClient.from_profile(profile_name) as client:
    ...
```

Or override explicitly when the situation calls for an ad-hoc connection:

```
async with KneoClient.from_profile(url="https://ad-hoc.example.com", api_key=tok) as client:
    ...
```

Explicit kwargs always win, so you can keep the TOML file as a baseline and override per-call.

Programmatic profile construction (when secrets come from a vault / secrets manager) skips `load_profile()` entirely:

```
from kneo_client import KneoClient
from kneo_client.core.auth import AuthScheme
from kneo_client.core.profiles import Profile

def profile_from_vault() -> Profile:
    secret = vault.get("kneo/prod")
    return Profile(
        name="prod",
        url=secret["url"],
        api_key=secret["api_key"],
        auth_scheme=AuthScheme.BEARER,
        timeout=30.0,
    )

async with KneoClient(profile_from_vault()) as client:
    ...
```

`Profile` is a frozen dataclass — pass it directly to `KneoClient(profile)`.

Inspecting a resolved profile

`KneoClient.profile` returns the `Profile` actually in use:

```
async with KneoClient.from_profile() as client:
    print(f"connected to {client.profile.url} as profile {client.profile.name!r}")
```

The `api_key` field is on the dataclass — handle it like any other secret. The redaction-aware logger in `kneo_client.core.logging` masks the key whenever it logs request / response headers, but **the dataclass itself is not redacted** when you print it directly.

If you do want to log a profile safely:

```
print(f"profile name={p.name!r} url={p.url!r} scheme={p.auth_scheme.value!r} timeout={p.timeout}")
```

...just exclude `p.api_key` from anything that goes to a log sink.

Profile errors

`ProfileError` covers four failure modes:

Trigger	Message pattern
Missing <code>url</code> after all sources	"profile 'X': 'url' is not set ..."
Missing <code>api_key</code> after all sources	"profile 'X': 'api_key' is not set ..."
Malformed TOML	"failed to parse <path>: ..."
Unknown auth scheme	"unknown auth_scheme '...'; expected one of: bearer, kneo_api_key"
Non-numeric <code>KNEO_TIMEOUT</code>	"\$KNEO_TIMEOUT must be a float, got '...'"

All five are explicit and name the offending source. Catch `ProfileError` (or `Exception` if you don't care which) at process startup to fail fast with a clear message rather than blowing up on the first call.

Default config path

`kneo_client.core.profiles.default_config_path()` returns the XDG-style default — `~/.config/kneo/client.toml` on Linux, `~/Library/Application Support/kneo/client.toml` on macOS, the appropriate `%APPDATA%\kneo\client.toml` on Windows. Resolved via `platformdirs.user_config_dir("kneo")`.

If you want a project-local TOML (committed to a repo, picked up by CI without needing a user-config), pass it explicitly:

```
from pathlib import Path
```

```
client = KneoClient.from_profile(config_file=Path(".kneo.toml"))
```

Idempotency and retries

Source: docs/user/idempotency_and_retries.md

This guide explains when `kneo-client` injects idempotency keys, how 409 mismatches surface, what the retry policy is, and how to customize either piece for non-default deployments.

What "idempotency and retries" mean in kneo-client

The Kneo Agent Platform's operational endpoints are designed for *safe retry*. The platform short-circuits a duplicate `POST` with the same `Idempotency-Key` header and identical payload — the second request replays the original response rather than re-executing the side effect.

`kneo-client` makes that safety automatic by:

1. Auto-injecting a fresh UUID4 `Idempotency-Key` on every `POST` (unless the caller supplies their own).
2. Retrying transient transport errors and the fixed set `{429, 502, 503, 504}` within a configurable `RetryPolicy`.
3. Honoring `Retry-After` on 429 responses (server's hint overrides the policy's computed delay).
4. Surfacing the platform's payload-mismatch behavior as `KneoIdempotencyMismatchError` so retry-with-wrong-payload bugs are loud, not silent.

Both pieces work together: idempotency keys make it *safe* to retry a `POST`; the retry policy *decides* when to retry. The default settings work for most callers; you can override either independently.

Idempotency keys, by default

On every `POST` the transport sends, it adds:

- `Idempotency-Key: <fresh UUID4>` — unless the caller passes `idempotency_key=<string>` on the method call.

The platform's contract:

- **Same key + identical payload** → server short-circuits and returns the original response. The retry is effectively a replay.
- **Same key + different payload** → server returns HTTP 409. The client surfaces this as `KneoIdempotencyMismatchError`.
- **Different key** → server treats this as a new request and executes the side effect.

The auto-generated key is a fresh UUID4 per request. Collisions are not a real concern (UUID4 has 122 bits of randomness), so by default each call is independent — retries beyond the transport's own loop won't be deduplicated by the server.

When to supply your own key

The auto-generated key is fine for one-shot calls. Supply your own when:

- **You're retrying outside the transport's loop.** The transport retries within `RetryPolicy.max_attempts` for transient failures, but if your *application* catches an error and retries (e.g. a job runner re-invoking after a process restart), pass the same key on both attempts so the platform dedupes. The transport's retries already share the key.
- **You want cross-process correlation.** Two services submitting the same logical request can dedupe by agreeing on the key (e.g. derive it from a hash of the request payload + a request ID from your application).
- **You're testing.** A stable key makes test fixtures deterministic.

```
from kneo_client.core.idempotency import new_idempotency_key

key = new_idempotency_key()
body = {"spec_id": "my-spec"}

# First attempt – succeeds normally
run = await client.platform.runs.create(body, idempotency_key=key)

# ... later, retrying after a transient outage outside the transport's loop:
run = await client.platform.runs.create(body, idempotency_key=key)
# → returns the first run (same response, no new side effect)
```

Constraints on caller-supplied keys (validated by the client before sending):

- **Non-empty** — an empty string raises `ValueError`.
- **At most 256 characters** (`MAX_KEY_LENGTH`). The platform enforces this; the client validates locally to fail faster.

Catching 409 mismatch

A 409 with an idempotency key set means the *same key was reused with a different payload* — almost always a caller bug. Surface it loudly:

```
from kneo_client.core.errors import KneoIdempotencyMismatchError

try:
```



```

await client.platform.runs.create(payload, idempotency_key=key)
except KneoIdempotencyMismatchError as exc:
    raise RuntimeError(
        f"Idempotency-Key {exc.idempotency_key!r} was reused with a different payload. "
        f"Either generate a new key for the new request or fix the payload drift."
    ) from exc

```

The exception carries the key as sent (`exc.idempotency_key`), the platform's error body (`exc.body`), the server-assigned request ID for log correlation (`exc.request_id`), and the HTTP status (`exc.status == 409`).

Note that `KneoIdempotencyMismatchError` is a *subclass* of `KneoConflictError`, so a generic `except KneoConflictError` will also catch it. If you only care about non-mismatch conflicts (resource state collisions, optimistic-lock failures, etc.), catch `KneoIdempotencyMismatchError` first.

The retry policy

`RetryPolicy` is a frozen dataclass describing when and how long to wait between attempts. The transport applies it; the policy itself does no I/O.

The default policy:

```

RetryPolicy(
    max_attempts=3,      # up to 3 total attempts
    base_delay=0.2,      # ~0.2s before attempt 2
    max_delay=30.0,      # cap on the computed delay
    jitter=0.1,          # 10% jitter applied to the computed delay
)

```

Delay sequence (without jitter): attempt 1 → no delay, attempt 2 → `base_delay`, attempt 3 → `2 × base_delay`, ..., capped at `max_delay`. With `jitter=0.1`, a 1-second delay becomes uniformly distributed in `[0.9, 1.1]`.

`Retry-After` from a 429 response overrides the computed delay verbatim. The server's hint is authoritative; no jitter is applied on top.

When retries fire

The transport retries on:

- Transport-level errors from `httpx` — DNS resolution failures, connect failures, TLS handshakes, read timeouts.
- HTTP **429** (rate limited), **502** (bad gateway), **503** (service unavailable), **504** (gateway timeout). Other 4xx and 5xx status codes do *not* trigger a retry — those typically indicate

caller errors or non-transient server problems that retrying won't fix.

The retryable status set is `RETRYABLE_STATUS_CODES = frozenset({429, 502, 503, 504})` — a module constant, intentionally not configurable. If your platform deployment legitimately returns transient `500`s, fix the deployment; we'd rather diagnose the root cause than open the gate.

Retries fire only for:

- **Idempotent verbs:** `GET`, `HEAD`, `OPTIONS`, `PUT`, `DELETE`.
- **`POST` with an `Idempotency-Key`** — i.e. always, since the transport auto-injects one on every `POST`.

This is why auto-injection matters: it's what makes `POST` retries safe.

Customizing the policy

Pass a custom policy when constructing the client:

```
from kneo_client import KneoClient
from kneo_client.core.profiles import load_profile
from kneo_client.core.retries import RetryPolicy

profile = load_profile()

# Aggressive retries for a flaky network
policy = RetryPolicy(max_attempts=8, base_delay=0.5, max_delay=60.0)
client = KneoClient(profile, retry_policy=policy)

# Flat delay (no exponential growth)
policy = RetryPolicy(max_attempts=3, base_delay=2.0, max_delay=2.0, jitter=0)
client = KneoClient(profile, retry_policy=policy)
```

To disable retries entirely (useful in tests that want to see the first failure surface immediately):

```
client = KneoClient(profile, retry_policy=RetryPolicy(max_attempts=1))
```

Constraints on `RetryPolicy` parameters (validated at construction):

- `max_attempts ≥ 1`
- `base_delay ≥ 0`
- `max_delay ≥ base_delay`
- `0 ≤ jitter ≤ 1`

Invalid values raise `ValueError` immediately — you find out at startup, not on the first retry.

What surfaces when retries exhaust

If all retries fail, the client raises the appropriate typed exception based on the *final* attempt's outcome:

Final attempt failed because...	Exception raised
HTTP 429	<code>KneoRateLimited</code> (carries <code>.retry_after</code>)
HTTP 502 / 503 / 504	<code>KneoServerError</code>
Transport error (DNS, connect, TLS, read)	<code>KneoNetworkError</code>

Intermediate retry attempts are logged at `INFO` level under the `kneo_client.transport` logger:

```
INFO:kneo_client.transport:status 503 on attempt 1; sleeping 0.20s
INFO:kneo_client.transport:status 503 on attempt 2; sleeping 0.40s
```

If you want to see the retry behavior in your application logs, set the logger level:

```
import logging
logging.basicConfig(level=logging.INFO)
logging.getLogger("kneo_client.transport").setLevel(logging.INFO)
```

Bypassing the auto-injection

The transport auto-injects an `Idempotency-Key` on every `POST` *unconditionally*. If for some reason you need to send a `POST` without one (very unusual — the platform's contract assumes the header is present), drop to the transport directly and supply explicit headers:

```
# Standard call – transport adds Idempotency-Key automatically
await client.platform.runs.create(body)

# No-auto-inject path (you take responsibility for deduplication)
await client._transport.request("POST", "/v1/runs", json=body, headers={"Idempotency-Key":
"your-deterministic-key"})
```

You almost never want this. Auto-injection is the right default; this is documented mostly so you know the escape hatch exists.

Putting it together — a robust pattern

```

from kneo_client import KneoClient
from kneo_client.core.errors import (
    KneoIdempotencyMismatchError, KneoNetworkError, KneoServerError,
)
from kneo_client.core.idempotency import new_idempotency_key

async def create_run_robust(client: KneoClient, body: dict) -> str:
    key = new_idempotency_key() # one key, used across application-level retries

    for attempt in range(1, 4):
        try:
            run = await client.platform.runs.create(body, idempotency_key=key)
            return run.run_id

        except KneoIdempotencyMismatchError as exc:
            # Caller bug – body changed between attempts. Don't retry.
            raise RuntimeError(f"payload drift on key {exc.idempotency_key!r}") from exc

        except (KneoNetworkError, KneoServerError) as exc:
            # The transport already retried within its policy; we add an outer
            # guard for application-level recovery (e.g. across process restarts).
            if attempt == 3:
                raise
            await asyncio.sleep(2 ** attempt)

    raise AssertionError("unreachable")

```

In practice the transport's built-in retries are sufficient for most use cases; the outer loop above is for the rare case where you want application-level retry behavior (e.g. survive a process restart while the platform is temporarily unreachable).

List-method results

Source: docs/user/list_results.md

Every `.list()` -style method on `kneo_client` returns a typed wrapper around the server response. There are two wrapper classes — chosen by the underlying server-response shape, not by "does this paginate":

- **Page[T]** for *list-shaped* responses (runs, checkpoints, trace events, human tasks, audit events).
- **Map[K, V]** for *dict-keyed* responses (credentials inventory, per-environment policies).

Both live in `kneo_client.core.results` and are exported from `kneo_client.core`. The free-function `iterate_all()` async iterator is also there and walks pages for `Page` returns whose endpoint supports it.

Per-method return shapes

Method	Returns	Server-side pagination
<code>client.platform.runs.list()</code>	<code>Page[Run]</code>	Full <code>limit / offset / total</code>
<code>client.platform.runs.checkpoints(run_id)</code>	<code>Page[Checkpoint]</code>	Full <code>limit / offset / total</code>
<code>client.platform.runs.trace(run_id)</code>	<code>Page[TraceEvent]</code>	Full <code>limit / offset / total</code>
<code>client.platform.human_tasks.list()</code>	<code>Page[HumanTask]</code>	Full <code>limit / offset / total</code>
<code>client.platform.audit.list()</code>	<code>Page[AuditEvent]</code>	Limit only (no <code>offset / total</code> ; see below)
<code>client.platform.credentials.list()</code>	<code>Map[str, Credential]</code>	None — whole map returned in one call
<code>client.platform.policies.environment_list()</code>	<code>Map[str, EnvironmentPolicy]</code>	None — whole map returned in one call

The item / value types (`Run`, `Checkpoint`, `Credential`, etc.) are the generated models from `kneo_client._generated.models`. Several are *open* objects in the pinned spec (no declared fields),

in which case fields are reached via bracket lookup — `run["run_id"]` rather than `run.run_id`. The [API Reference](#) lists which per-endpoint item models are typed vs open.

Page[T]

```
@dataclass(frozen=True)
class Page(Generic[T]):
    items: list[T]
    total: int | None = None
    limit: int | None = None
    offset: int | None = None
    sort_by: str | None = None
    sort_order: str | None = None

    @property
    def count(self) -> int: ...          # = len(items)

    @property
    def has_more(self) -> bool: ...      # False when offset/total are None

    def __iter__(self) -> Iterator[T]: ...
    def __len__(self) -> int: ...
    def __getitem__(self, index: int) -> T: ...
```

Metadata fields are Optional — an endpoint that doesn't echo a particular field (audit) leaves it as `None` rather than carrying a synthesized value. `has_more` derives sensibly: `False` whenever `offset` or `total` is `None`.

Iterating one page

`Page` is a sequence; iterate it directly:

```
page = await client.platform.runs.list(status="running", limit=50)
print(f"got {page.count} of {page.total} runs")
for run in page:
    print(run["run_id"])
```

Indexing and `len` work as expected:

```
first_run = page[0]
total_on_this_page = len(page)
```

Walking all pages: `iterate_all()`

For the four fully-paginating endpoints, `iterate_all()` walks pages transparently. Pass a `fetch_page(limit, offset)` callable that returns the next `Page`:

```
from kneo_client.core.results import iterate_all

async def fetch(limit: int, offset: int):
    return await client.platform.runs.list(limit=limit, offset=offset, status="running")

async for run in iterate_all(fetch, page_size=200):
    print(run["run_id"])
```

`iterate_all()` does three things on your behalf:

1. **Clamps `page_size` to `MAX_PAGE_SIZE = 1000`** (the platform's hard upper bound). Asking for more silently downsizes.
2. **Walks `offset` automatically** — each page's `offset` plus its `count` becomes the next page's starting position.
3. **Stops when `has_more` is `False`** — either because the server has no more data (`offset + count ≥ total`) or because the server doesn't echo enough metadata to know (audit, see next section).

The audit case (`limit`-only, gracefully degraded)

`audit.list()` is list-shaped but the platform endpoint accepts only `limit` — no `offset` — and the response echoes only `count + events`, not `limit`, `offset`, `total`, or sort metadata. The wrapper returns a `Page` whose Optional fields are `None`:

```
page = await client.platform.audit.list(event_type="run.created", limit=200)
assert page.total is None
assert page.offset is None
assert page.has_more is False # server doesn't tell us
for event in page:
    print(event)
```

`iterate_all()` on audit fetches once and exits because `has_more` is `False` — gracefully degraded, not lying. There are no further pages the client can reach via this endpoint; use the `event_type` / `run_id` filters to narrow the result set.

Choosing a `page_size`

Rules of thumb for the fully-paginating endpoints:

- **100–200** for most interactive flows — snappy first-byte latency.
- **500–1000** for back-end exports — fewer requests, higher per-request cost on retry.

- **< 50** when per-item downstream processing is slow and you want to start producing output sooner.

The hard ceiling is `MAX_PAGE_SIZE = 1000`; `iterate_all()` clamps for you.

Map[K, V]

```
@dataclass(frozen=True)
class Map(Generic[K, V]):
    items: Mapping[K, V]

    @property
    def count(self) -> int: ...          # = len(items)

    def __getitem__(self, key: K) -> V: ...
    def __iter__(self) -> Iterator[V]: ...    # iterates values
    def __len__(self) -> int: ...
    def __contains__(self, key: object) -> bool: ...

    def get(self, key: K, default: V | None = None) -> V | None: ...
    def keys(self) -> KeysView[K]: ...
    def values(self) -> ValuesView[V]: ...
```

`Map` covers the two endpoints whose responses are keyed maps rather than item lists. The whole collection comes back in one call — there is no `limit` / `offset` to think about.

```
inv = await client.platform.credentials.list()
print(f"{inv.count} credential references")
for credential in inv:                                # iterates values
    print(credential["provider"])

aws = inv["cred-aws"]                                # bracket lookup
if "cred-missing" in inv:                             # membership test
    ...

for cred_id in inv.keys():                             # explicit keys view
    ...

# Pair iteration uses the underlying Mapping when needed:
for cred_id, body in inv.items.items():
    print(cred_id, body["provider"])
```

Iteration yields *values*, not keys — consistent with `Page` iterating items. Use `.keys()` or `.items.items()` if you need keys.

Filters + paging

Resource-specific filter kwargs compose with paging kwargs:

```
# Just the failures, walked in pages of 200
async def fetch(limit, offset):
    return await client.platform.runs.list(
        status="failed", limit=limit, offset=offset
    )

async for run in iterate_all(fetch, page_size=200):
    ...

# Audit events for one run (single fetch – audit doesn't paginate)
events = await client.platform.audit.list(run_id="r1", limit=500)
```

Drop to the raw response

If you need a field the wrapper doesn't expose — for example the `run_id` echo on `CheckpointListResponse` / `TraceResponse`, or a field the platform adds in a newer release — bypass the wrapper and call the transport directly:

```
from kneo_client._generated.models.checkpoint_list_response import (
    CheckpointListResponse,
)

resp = await client._transport.request(
    "GET", f"/v1/runs/{run_id}/checkpoints", params={"limit": 100}
)
raw = CheckpointListResponse.from_dict(resp.json())
print(raw.run_id, raw.checkpoints, raw.total)
```

`_transport` is the underlying `Transport`; the leading underscore signals it's an escape hatch, not part of the recommended surface. Anything you build on top of it tracks the generated layer directly and won't follow public-API compatibility — re-evaluate at each `kneo_serv` pin bump.

Auto-walking is *not* on the roadmap

`client.platform.runs.list_all()` returning an async iterator — i.e. auto-paged at the adapter layer — is intentionally *not* planned. The current shape (list methods return one `Page`; `iterate_all()` walks pages explicitly) keeps the per-call cost transparent and lets the caller decide when to stop. An accidental wide filter shouldn't walk millions of items behind the caller's back.

If you want a one-liner in your own code, write a small helper around `iterate_all()` and the relevant `.list()` callable — the pattern is identical for every fully-paginating endpoint.

Polling and waiting

Source: docs/user/polling_and_waiting.md

Two helpers on `RunsClient` cover the two common run-lifecycle waiting patterns:

- `wait_for_completion()` — block until a run reaches a terminal status, returning the final status. Best when you only care about the outcome and will inspect the trace afterwards.
- `tail_trace()` — stream trace events as they arrive, returning an async iterator. Best when you want to surface progress live (Studio editors, CLIs, dashboards).

Both follow the same polling-with-status-check pattern under the hood; they differ in what they yield to the caller.

`wait_for_completion()`

```
status = await client.platform.runs.wait_for_completion(
    run_id,
    poll_interval=1.0,  # seconds between GETs to /v1/runs/{run_id}
    timeout=600,        # total budget; None to wait indefinitely
)
print(status.status)    # "completed" | "failed" | "cancelled" | ...
```

The default `terminal_statuses` set is `{"completed", "failed", "cancelled"}`. Pass a custom set to treat additional states as terminal:

```
# Stop as soon as the run pauses for human review.
status = await client.platform.runs.wait_for_completion(
    run_id,
    terminal_statuses={"paused_human_review"},
)
```

Raises `TimeoutError` if the deadline elapses before any terminal status is reached.

`tail_trace()`

```
async for event in client.platform.runs.tail_trace(run_id, poll_interval=1.0):
    print(event["event_type"], event.get("payload"))
```

Internally `tail_trace` walks `/v1/runs/{run_id}/trace` with ascending offset and yields each event as it lands. When the run reaches a terminal status (configurable, same defaults as

`wait_for_completion`) the helper does one final drain pass to capture events emitted between the last poll and the status transition, then returns.

Key arguments (all optional):

Arg	Default	Notes
<code>start_offset</code>	0	Resume tailing from a known position.
<code>page_size</code>	100	limit per <code>/v1/runs/{run_id}/trace</code> fetch.
<code>poll_interval</code>	1.0	Seconds to sleep when no new events arrived.
<code>timeout</code>	None	Total budget for reaching terminal status. Does not apply during the post-terminal drain — that always runs to completion.
<code>terminal_statuses</code>	<code>{"completed", "failed", "cancelled"}</code>	Override to treat other states as terminal.
<code>event_type</code>	None	Filter passed through to <code>trace</code> .

Raises `TimeoutError` if the deadline elapses while waiting for a terminal status. Events yielded so far stay yielded — the caller has already consumed them.

When to use which

Want	Use
Just the final outcome	<code>wait_for_completion()</code>
Final outcome + the full trace at the end	<code>wait_for_completion()</code> then <code>runs.trace()</code>
Events as they arrive (live UI / log tail)	<code>tail_trace()</code>
First N events of a finished run	<code>runs.trace(limit=N)</code> directly
Resume an interrupted tail	<code>tail_trace(start_offset=...)</code>

Polling cadence

A rule of thumb: pick `poll_interval` so that *p99 perceived latency* roughly equals `poll_interval / 2`.

- **Interactive UIs (Studio, dashboards)** — 0.5 to 1.0 seconds. Snappy enough for human attention.
- **Operational scripts (CI, scheduled jobs)** — 2.0 to 5.0 seconds. Reduces request volume for long-running jobs where snappy feedback isn't needed.
- **Long-running background workers** — 10.0+ seconds. Pair with a generous `timeout`.

Don't go below 0.5 seconds without a specific reason — at that rate you're spending more on request overhead than you save in latency, and the platform's retry / backoff machinery is also factoring in.

Custom terminal statuses

The platform reports several non-final statuses (`running`, `queued`, `paused_human_review`) that the default terminal set excludes.

The most common override is **treating human-review pauses as terminal** — useful when an operator-facing tool wants to surface "this run needs your attention" rather than continue polling:

```
status = await client.platform.runs.wait_for_completion(
    run_id,
    terminal_statuses={"completed", "failed", "cancelled", "paused_human_review"},
)
if status.status == "paused_human_review":
    # Surface a UI prompt; resume via the human-tasks API.
    ...
```

`tail_trace()` accepts the same kwarg with the same semantics.

TimeoutError handling

Both helpers raise `TimeoutError` rather than returning a partial result, because returning a "not yet terminal" status silently is easy to misuse. Pattern:

```
try:
    status = await client.platform.runs.wait_for_completion(
        run_id, timeout=300
    )
except TimeoutError:
    # Decide whether to cancel, escalate, or keep waiting with a fresh budget.
```

```
current = await client.platform.runs.get(run_id)
if current.status == "running":
    await client.platform.runs.cancel(run_id)
raise
```

For `tail_trace`, events yielded *before* the timeout stay consumed:

```
seen: list = []
try:
    async for event in client.platform.runs.tail_trace(run_id, timeout=60):
        seen.append(event)
except TimeoutError:
    print(f"timed out with {len(seen)} events streamed")
    raise
```

Error handling

Source: <docs/user/errors.md>

This guide explains the typed exception hierarchy `kneo-client` raises, what each exception carries, and how to write robust catch blocks for the common operational shapes.

What "errors" mean in kneo-client

Every failure — at any layer — surfaces as a typed exception derived from `KneoError`. There is no `(ok, err)` tuple return, no `Response[T]` wrapper, no `errno` field. The standard Python `try / except` flow is the *only* error-handling shape.

Each exception carries enough context to:

- **Log the failure with traceability** — every exception has `.request_id` (the server-assigned correlation ID) and, for `POST` failures, `.idempotency_key`.
- **Branch on the operational meaning** — the exception class encodes "what went wrong" (auth vs. permission vs. server outage vs. network).
- **Read the server's reason** — `.body` is the parsed JSON the platform returned (or raw text when the response wasn't JSON).
- **Decide whether to retry, escalate, or surface** — combined with `.status` and the exception type, you can route the failure programmatically.

Hierarchy

```
KneoError
├─ KneoNetworkError           # DNS / connect / TLS / read timeout – wrapped from
httpx.HTTPError
├─ KneoAuthError             # HTTP 401 – missing or invalid API key
├─ KneoPermissionError       # HTTP 403 – key valid but lacks the required scope
├─ KneoNotFoundError         # HTTP 404 – resource does not exist
├─ KneoConflictError         # HTTP 409 (generic)
│   └─ KneoIdempotencyMismatchError # HTTP 409 with payload mismatch on a replayed
Idempotency-Key
├─ KneoRateLimited           # HTTP 429 (carries .retry_after)
└─ KneoServerError           # HTTP 5xx – server-side failure
```

`KneoIdempotencyMismatchError` is a *subclass* of `KneoConflictError` (catching `KneoConflictError` also catches the mismatch case). Everything else is parallel.

What every exception carries

```
class KneoError(Exception):
    status: int | None          # HTTP status, or None for transport-level failures
    body: Any                  # Parsed JSON dict, raw text, or None
    request_id: str | None      # X-Request-ID echoed by the server
    idempotency_key: str | None # Idempotency-Key sent on the failing request (POSTs)
```

`KneoRateLimited` adds one field:

```
class KneoRateLimited(KneoError):
    retry_after: float | None # Seconds parsed from the Retry-After header
```

All other subclasses inherit from `KneoError` without adding fields.

Status code → exception mapping

HTTP status	Exception	When
401	<code>KneoAuthError</code>	Missing or invalid API key. Re-check the profile resolution chain.
403	<code>KneoPermissionError</code>	API key is valid but the platform won't authorize this operation. Check the key's scopes / role.
404	<code>KneoNotFoundError</code>	The resource (run, spec, environment, etc.) doesn't exist. Often a stale ID.
409	<code>KneoConflictError</code>	Generic conflict — resource state, optimistic-lock failure, etc.
409 with idempotency key	<code>KneoIdempotencyMismatchError</code> or	Same key reused with a different payload — see idempotency .
429	<code>KneoRateLimited</code>	Rate limit hit. <code>.retry_after</code> carries the server's hint.
5xx	<code>KneoServerError</code>	Server-side failure. The transport already retried within

HTTP status	Exception	When
		its policy (for 502/503/504); a <code>KneoServerError</code> reaching your code means retries exhausted.
Other (1xx / 3xx / 4xx that aren't above)	<code>KneoError</code> (base)	Catch-all for unmodeled statuses.
Connection / DNS / TLS / read timeout	<code>KneoNetworkError</code>	Transport-level failure. The transport already retried for transient errors; a <code>KneoNetworkError</code> reaching your code means retries exhausted.

Catching patterns

Catch broadly, log richly

The most common pattern — log the full context, then decide whether to re-raise:

```
from kneo_client.core.errors import KneoError

try:
    run = await client.platform.runs.create(payload)
except KneoError as exc:
    log.error(
        "create_run failed status=%s request_id=%s idempotency_key=%s body=%r",
        exc.status,
        exc.request_id,
        exc.idempotency_key,
        exc.body,
    )
    raise
```

The `request_id` is the link to the platform's audit events — pass it along when reporting a problem to the platform operators.

Branch on specific status

When the operational meaning matters (auth flow, retry decision, user-visible error message):

```

from kneo_client.core.errors import (
    KneoAuthError,
    KneoNotFoundError,
    KneoRateLimited,
    KneoServerError,
)

try:
    run = await client.platform.runs.get(run_id)
except KneoAuthError:
    print("API key is missing, invalid, or revoked.")
    raise
except KneoNotFoundError:
    print(f"run {run_id!r} does not exist.")
    return None
except KneoRateLimited as exc:
    print(f"rate-limited; server suggests waiting {exc.retry_after}s")
    await asyncio.sleep(exc.retry_after or 10)
    raise
except KneoServerError as exc:
    log.error("platform 5xx (after retries): %s", exc.body)
    raise

```

Order matters: catch more specific exceptions first (`KneoNotFoundError` before `KneoError`).

Handle idempotency-key mismatches loudly

A 409 with an idempotency key set means the *same key was reused with a different payload*. This is almost always a caller bug — surface it explicitly:

```

from kneo_client.core.errors import KneoIdempotencyMismatchError

try:
    await client.platform.runs.create(payload, idempotency_key=key)
except KneoIdempotencyMismatchError as exc:
    raise RuntimeError(
        f"Idempotency-Key {exc.idempotency_key!r} was reused with a different payload. "
        f"Either generate a new key for the new request or fix the payload drift."
    ) from exc

```

See [Idempotency and retries](#) for the full story on how / when this happens.

Don't catch successful-status branches

Methods on the platform / agent clients return parsed response models on success and *raise* on failure. There is no "ok / err" branching at the call site. Wrap the *call* in `try` / `except` , not the return value:

```
# Right
try:
    run = await client.platform.runs.create(payload)
    process(run)
except KneoError:
    ...

# Wrong – runs.create never returns None / False on failure; it raises
result = await client.platform.runs.create(payload)
if result is None: # never happens
    ...
```

Transport errors and network troubleshooting

`KneoNetworkError` covers everything below the HTTP layer: DNS resolution, TCP connect failures, TLS handshakes, read timeouts, connection resets. It wraps the underlying `httpx.HTTPError` as the cause — `exc.__cause__` is the original `httpx` exception if you need to inspect it.

The transport retries these automatically within `RetryPolicy.max_attempts` for transport errors and for HTTP 429 / 502 / 503 / 504. So a `KneoNetworkError` reaching your code means **all retries exhausted**:

```
from kneo_client.core.errors import KneoNetworkError

try:
    health = await client.platform.health.readyz()
except KneoNetworkError as exc:
    print(f"could not reach the platform: {exc}")
    # Treat as a hard dependency outage; don't pretend the call succeeded.
    raise
```

If you want to see the retry behavior in your logs, set the `kneo_client.transport` logger to `INFO`:

```
import logging
logging.getLogger("kneo_client.transport").setLevel(logging.INFO)
# → INFO kneo_client.transport: transport error on attempt 1; sleeping 0.20s: ...
```

Timeouts

`Profile.timeout` (default 30s, settable per-profile in `~/.config/kneo/client.toml` or via the `KNEO_TIMEOUT` env var) becomes `httpx`'s single-number timeout, which is shorthand for *all four* of `httpx`'s timeout dimensions: connect, read, write, and pool. So a `timeout=30.0` means each of those phases independently must complete within 30s, not that the whole request must finish within 30s.

A read timeout while waiting for a slow `runs.create` or `agent.specs.compile` response surfaces as `KneoNetworkError(__cause__=httpx.ReadTimeout(...))`. The transport's retry policy applies — if you're seeing these reach your code, increase `timeout` (e.g. `KNEO_TIMEOUT=120`) rather than raising `RetryPolicy.max_attempts`, since retrying a slow request just rebuilds the same wait.

For finer control (e.g. a long read budget but a short connect budget), inject a custom `httpx` client with a `httpx.Timeout` instance:

```
import httpx
from kneo_client import KneoClient

custom = httpx.AsyncClient(
    base_url=profile.url,
    timeout=httpx.Timeout(connect=5.0, read=120.0, write=30.0, pool=5.0),
)
client = KneoClient(profile=profile, http_client=custom)
# Caller owns `custom`'s lifecycle when passing http_client.
```

TLS verification

`httpx` verifies server certificates by default against the `certifi` CA bundle, and that's what `kneo-client` uses with no extra configuration. Three common operational variations:

- **Custom CA bundle** (corporate proxy with an internal CA): pass `verify="/path/to/ca.pem"` to a custom `httpx.AsyncClient`.
- **Disable verification** (development against a self-signed staging instance, *not for prod*): `verify=False`. Don't do this against an internet-reachable platform.
- **Client certificates** (mTLS-protected staging): `cert=("/path/cert.pem", "/path/key.pem")`.

All three flow through the `http_client=` injection point shown in the *Timeouts* example. There is no `kneo-client`-level wrapper for these — the configuration surface is `httpx`'s.

A failing TLS handshake surfaces as `KneoNetworkError(__cause__=httpx.ConnectError(...))` with a message like `[SSL: CERTIFICATE_VERIFY_FAILED]`. Check the `__cause__` for the precise OpenSSL error code.

HTTP proxies

`httpx` automatically picks up `HTTPS_PROXY`, `HTTP_PROXY`, and `NO_PROXY` from the environment — `kneo-client` inherits that behavior with no extra wiring. So:

```
HTTPS_PROXY=http://proxy.corp.example.com:8080 python my_script.py
```

works out of the box. For an explicit proxy URL set in code (without touching env vars), inject a custom httpx client:

```
custom = httpx.AsyncClient(
    base_url=profile.url,
    proxy="http://proxy.corp.example.com:8080",
)
```

A proxy that returns 502 or refuses the upstream connection surfaces as `KneoServerError` (if the proxy returns a real HTTP error) or `KneoNetworkError` (if the connection itself fails). Check `__cause__` to distinguish.

Inspecting the underlying httpx exception

For diagnosing intermittent network issues, the original httpx exception carries more context than `KneoNetworkError`'s string. Walk the cause chain:

```
try:
    await client.platform.runs.list()
except KneoNetworkError as exc:
    cause = exc.__cause__
    print(f"kneo wrapper: {exc}")
    print(f"httpx cause: {type(cause).__name__}: {cause}")
    # Common httpx exception types you'll see here:
    # httpx.ConnectTimeout    - couldn't connect within the connect budget
    # httpx.ReadTimeout       - server stopped responding mid-read
    # httpx.ConnectError      - DNS / TCP / TLS handshake failed
    # httpx.RemoteProtocolError - connection reset, partial response, etc.
    # httpx.ProxyError        - proxy-specific failure
```

When opening a support ticket for an intermittent failure, including the `__cause__`'s type and message lets the operator distinguish connection-pool exhaustion from a stuck upstream from a proxy misconfiguration.

Building error messages for users

The exceptions are designed for *internal* error handling, not for user-facing messages. If you're surfacing platform errors to end users (in a dashboard UI, a CLI prompt, etc.), build a friendly message from the exception's attributes rather than printing `str(exc)`:

```
def user_message(exc: KneoError) -> str:
    if isinstance(exc, KneoAuthError):
        return "Your API key is invalid. Please check your credentials."
    if isinstance(exc, KneoPermissionError):
```

```

    return "You don't have permission to perform this action."
if isinstance(exc, KneoNotFoundError):
    return "The requested item could not be found."
if isinstance(exc, KneoRateLimited):
    wait = exc.retry_after or 60
    return f"Too many requests. Please try again in {int(wait)}s."
if isinstance(exc, KneoServerError):
    return f"Server error (request {exc.request_id}). Please report this."
if isinstance(exc, KneoNetworkError):
    return "Could not reach the server. Check your network connection."
return f"Unexpected error: {exc}"

```

Why a typed hierarchy

A single `KneoError` would force callers to inspect `.status` everywhere they want to branch. A flat enum of error codes would lose the natural `isinstance` ergonomics. The hierarchy lets you catch broadly (`except KneoError`) for logging and narrowly (`except KneoAuthError`) for recovery — without losing the underlying response context, which stays attached to the exception instance.

Subclasses are added when the platform introduces a new HTTP status that warrants its own catch site, and not before. The set above is sufficient for `/v1` as it stands at `kneo_serv 0.4.0`.

Reference

Helper	Where	What it does
<code>from_response(response, *, idempotency_key=None)</code>	<code>kneo_client.core.errors</code>	Maps an <code>httpx.Response</code> to the appropriate <code>KneoError</code> subclass. Used internally by <code>Transport</code> ; rarely called directly.
<code>KneoError(message, *, status, body, request_id, idempotency_key)</code>	<code>kneo_client.core.errors</code>	The base. All subclasses share this constructor signature (except <code>KneoRateLimited</code> , which adds <code>retry_after</code>).

Compatibility matrix

Source: <docs/user/compatibility.md>

This guide tells you which `kneo-client` release supports which `kneo_serv` platform version, what forward and backward compatibility mean in practice, and how to use the drop-to-transport escape hatch when you need access to an endpoint the current `kneo-client` release doesn't wrap yet.

What "compatibility" means in kneo-client

The Kneo Agent Platform's `/v1` HTTP API is a stability boundary — a `kneo-client` release pinned to one `kneo_serv` minor works against any patch-level `kneo_serv` release on the same minor line, and against newer minors that don't break `/v1`.

The pinning is *explicit* and *committed*: <schemas/openapi.json> is a `/v1`-filtered copy of one specific `kneo_serv` release's published OpenAPI spec. Bumping the pin is a deliberate PR (via scripts/bump_schemas.py), reviewed in isolation, and never happens automatically. See [ADR-004](#) for the rationale.

Current matrix

<code>kneo-client</code>	Pinned to <code>kneo_serv</code>	Tested against	Python	Status
0.4.0	v0.4.0 (<code>info.version</code> 0.4.0)	<code>kneo_serv</code> 0.4.x line	<code>>=3.12</code> (3.12 / 3.13 / 3.14 in CI)	Current
0.3.0	v0.4.0 (<code>info.version</code> 0.4.0)	<code>kneo_serv</code> 0.4.x line	<code>>=3.12</code> (3.12 / 3.13 / 3.14 in CI)	Previous
0.2.0	v0.4.0 (<code>info.version</code> 0.4.0)	<code>kneo_serv</code> 0.4.x line	<code>>=3.12</code>	Older

The pinned `kneo_serv` version is recorded in <schemas/SOURCE.md> — that's the source of truth for which platform version generated the committed `_generated/` tree.

How pinning works in practice

The pin is the input to the generated layer. When you bump it:

1. `scripts/bump_schemas.py` fetches the new `openapi.json` from the target `kneo_serv` ref (or a local checkout).
2. The spec is filtered to `/v1` paths only — `kneo_serv` mounts every route at both `/v1/...` and `/...`; we drop the unprefix mounts.
3. The filtered spec replaces `schemas/openapi.json`, `schemas/SOURCE.md` is updated, and `_generated/` is regenerated.
4. The hand-rolled adapter layer (`platform/`, `agent/`) may need updates if endpoints were added, renamed, or had their shapes change. The contract test [tests/contract/test_path_coverage.py](#) catches both sides of that drift.

A `kneo-client` minor release ships exactly one `kneo_serv` pin. Patches don't change pins. A pin bump can land in any minor / patch — versioning is independent.

Forward compatibility — newer `kneo_serv` than the pin

`kneo-client` ships an explicit list of every `(method, path)` it wraps. A newer `kneo_serv` that adds endpoints will still work for everything `kneo-client` already wraps — you just won't have wrappers for the new endpoints until the next `kneo-client` release.

If you need access to a new endpoint before that release, drop to the raw transport:

```
async with KneoClient.from_profile() as client:
    # Call an endpoint that doesn't have a wrapper yet:
    resp = await client._transport.request("GET", "/v1/some/new/endpoint")
    payload = resp.json()
```

The `_transport` attribute is informally accessible (single-underscore prefix). It's not part of the documented stability surface, but the API has been stable since `0.1.0` and is unlikely to churn. The transport handles auth, retries, idempotency, request-ID injection, and error mapping just like the wrapped methods do — you just lose the typed response model.

Backward compatibility — older `kneo_serv` than the pin

`kneo-client X.Y.Z` is **not** guaranteed against `kneo_serv` releases older than its pin. Wrappers may rely on response fields that older `kneo_serv` versions don't emit, and the client's error mapping assumes the current platform error shape. A `KeyError` on `from_dict()` is the typical symptom — the wrapper expects a field that wasn't in the older platform's response.

If you have to talk to an older `kneo_serv`, pin to a matching `kneo-client` minor:

Need to talk to <code>kneo_serv</code> ...	Use <code>kneo-client</code>
<code>0.4.x</code>	<code>0.1.x</code> , <code>0.2.x</code> , or <code>0.3.x</code> (all pinned to <code>v0.4.0</code> ; pick <code>0.3.x</code> for the additive filter kwargs on <code>runs.checkpoints / credentials.list</code> , <code>0.2.x</code> for the <code>Page + Map</code> return-type ergonomics)

(More rows added as the project ships.)

Breaking changes

A breaking change in either direction triggers a major bump:

- **`kneo-client` major** — the public Python API (`kneo_client.*` namespace) changes incompatibly. Very rare.
- **`kneo_serv` major** — i.e., introduction of `/v2`. `kneo-client` may need a major bump if `/v1` is sunset; otherwise it ships a new minor that supports both.

Within a major, deprecated surfaces keep aliases for at least one minor. See [docs/dev/contributing.md](#) for the deprecation policy.

Verifying compatibility yourself

The simplest smoke is the bundled examples:

```
# Install a specific kneo-client version
python -m pip install "kneo-client==X.Y.Z"

# Point it at your kneo_serv instance
export KNEO_URL=https://your-kneo-serv.example.com
export KNEO_API_KEY=...

# Run the smoke (touches health, runs, audit, agent specs, and human tasks)
python -m kneo_client.examples.01_basic_run YOUR_SPEC_ID
```

The five [examples/](#) scripts collectively touch the platform health, runs, audit, agent spec, and human-task surfaces — enough to catch obvious incompatibilities in seconds.

For deeper validation, the [integration test suite](#) is env-gated; set `KNEO_TEST_URL` and `KNEO_TEST_API_KEY` and run `python -m pytest tests/integration -v`.

What the pin does *not* guarantee

The pin guarantees the *wire format* and the *path set* of `/v1` . It does not guarantee:

- **Provider availability** — whether your `kneo_serv` deployment has GPT-4 configured, an MCP server reachable, a specific runtime registered. The pin says nothing about deployment state.
- **Spec compatibility** — a spec that works on one `kneo_serv` may fail on another if the spec relies on a specific provider, tool, or platform version. Validate specs against the target environment with `client.agent.specs.validate(...)` .
- **Policy outcomes** — `policies.environment_*` queries return whatever policy is configured on the target deployment.

These are platform-deployment concerns, not client-library concerns. The client gives you a clean wire to the platform; what the platform allows is a separate dimension.